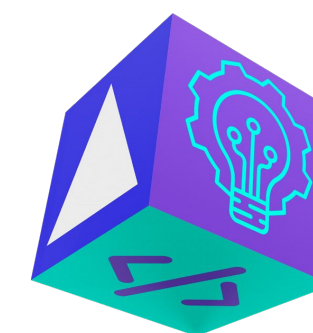
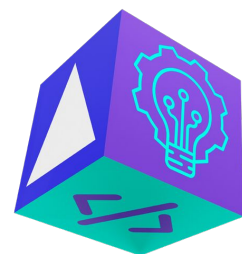




RESIDÊNCIA
EM SOFTWARE
BAHIA + TECNOLOGIA + EMPREENDEDORISMO

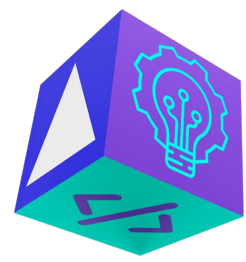
Trilha

**Desenvolvimento Front-End e
Conceitos de User Interface e
User Experience (UI/UX)**



Introdução ao Angular

1. O que é Angular?
2. Componentes
3. Templates
4. Pipes
5. Comunicação entre Componentes
6. Formulários Relativos



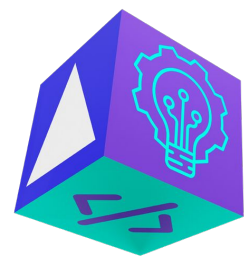
O que é Angular?

Angular é um framework web, mantido por uma equipe dedicada do Google, que oferece um amplo conjunto de ferramentas, APIs e bibliotecas para simplificar e agilizar o fluxo de trabalho de desenvolvimento.

Atualmente está na versão 18.

Pré-requisitos:

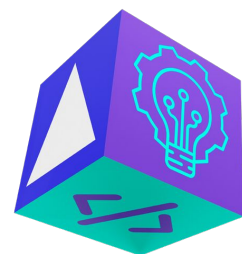
- JavaScript, inclusive conceitos de classes;
- Fundamentos de TypeScript;
- Fundamentos de decorators.



O que é Angular?

O Angular oferece um robusto conjunto de ferramentas e recursos, incluindo:

- Modelo de Componentes: Facilita a construção de interfaces de usuário modulares e reutilizáveis.
- Injeção de Dependência: Promove a modularidade e facilita o gerenciamento de dependências.
- Reatividade: Permite a criação de interfaces de usuário responsivas.
- Renderização no Lado do Servidor: Melhora o desempenho e a SEO.
- Ferramentas de Desenvolvimento: Inclui Angular CLI para automação de tarefas e Angular DevTools para depuração.



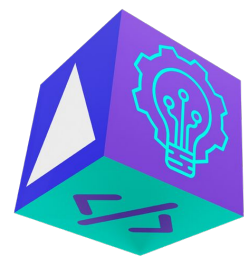
Componentes

Componentes fornecem a estrutura para organizar seu projeto em partes menores, mais fáceis de entender, com poucas responsabilidades, e mais simples de manter;

No Angular, um componente é definido a partir dos seguintes itens:

- Um decorator `@Component`;
- Um template HTML;
- Um estilo;
- Uma classe typescript para definição de comportamentos, estados, etc.

```
// Exemplo de componente
@Component({
  standalone: true,
  selector: "nome-componente",
  template: ` <li>Angular Essentials Guide</li> `,
  styles: `
    li {
      color: red;
    }
  `,
})
export class NomeComponente {
  /* O comportamento do componente é definido aqui */
}
```

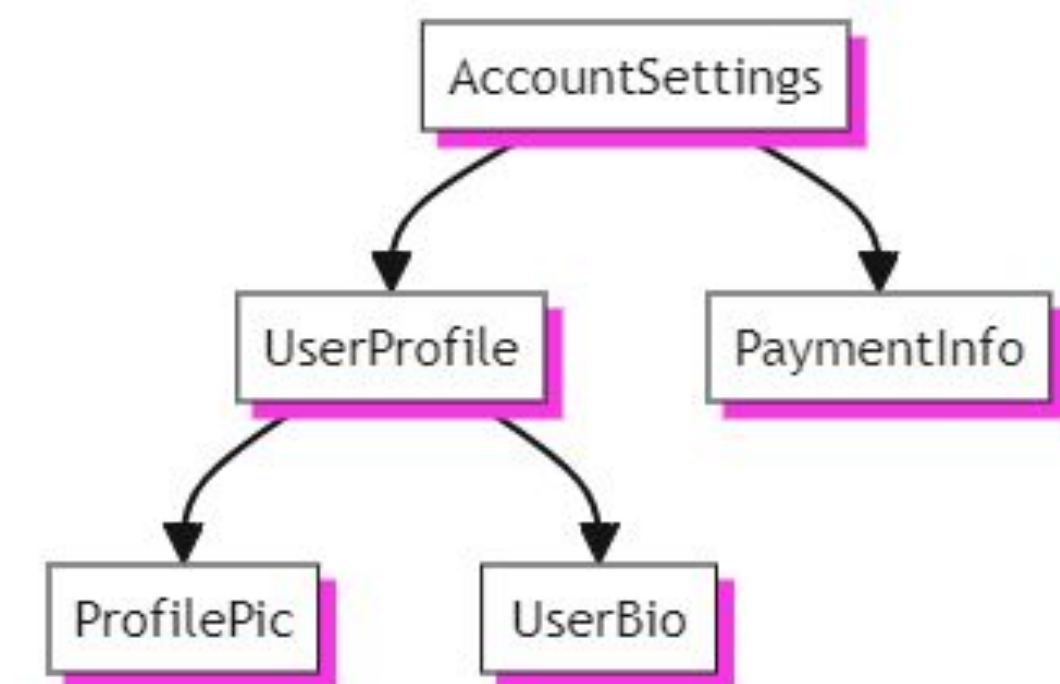


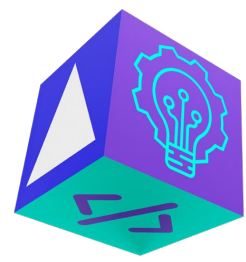
Componentes

Componentes podem ser compostos por outros componentes;

```
import { ButtonComponent } from './components/button/button.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [ButtonComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss'],
})
export class AppComponent {
  title = 'projeto-angular';
}
```





Componentes

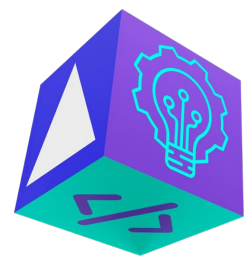
Estado

Estado refere-se ao gerenciamento e rastreamento de dados ou informações que são necessários para a aplicação funcionar corretamente.

O estado pode ser gerenciado diretamente dentro dos componentes, a partir dos atributos e métodos do objeto.

```
@Component({
  selector: 'app-todolist',
  standalone: true,
  imports: [],
  templateUrl: './todolist.component.html',
  styleUrls: ['./todolist.component.scss'],
})
export class TodoListComponent {
  taskTitle = '';
  isComplete = false;

  updateTitle(newTitle: string) {
    this.taskTitle = newTitle;
  }
}
```

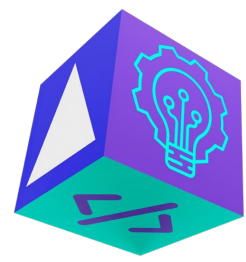


Templates

Um template é um fragmento de HTML que define a estrutura e o layout da interface do usuário de um componente.

O template é essencialmente a "view" do componente, onde você especifica como os dados devem ser exibidos e interagem com o usuário.

Ele pode incluir diretivas, bindings e outros elementos que tornam a interface dinâmica e interativa.



Templates

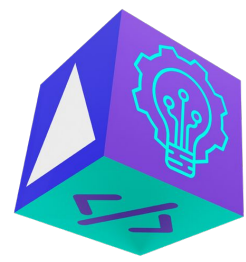
Data Binding

Permite a vinculação de dados entre o componente e o template, seja de forma unidirecional ou bidirecional.

Pode acontecer por:

- **Interpolação:** Quando você precisa exibir conteúdo dinâmico em seu modelo, o Angular usa a sintaxe de dupla chave, a fim de distinguir entre conteúdo estático e dinâmico.

```
@Component({  
  selector: "todo-list-item",  
  template: ` <p>Title: {{ taskTitle }}</p> `,  
})  
  
export class TodoListItem {  
  taskTitle = "Read cup of coffee";  
}
```



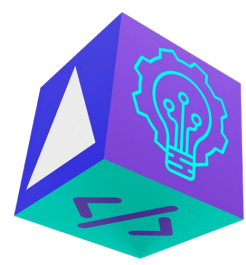
Templates

Data Binding

Pode acontecer por:

- **Property Binding:** Quando você precisa definir dinamicamente o valor das propriedades DOM padrão em um elemento HTML, a propriedade é envolto em colchetes para informar o Angular que o valor declarado deve ser interpretado como uma instrução JavaScript

```
@Component({
  selector: 'sign-up-form',
  template: `
    <button type="submit" [disabled]="formIsValid">Submit</button>
  `,
})
export class SignUpForm {
  formIsValid = true;
}
```



Templates

Data Binding

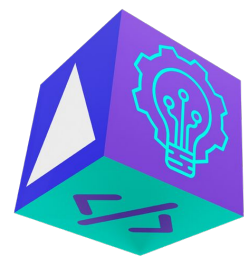
Pode acontecer por:

- **Event Binding:** Define dinamicamente a ação que será disparada quando ocorrer um determinado evento.

O evento é envolto em parênteses para informar o Angular que o valor declarado deve ser interpretado como uma instrução JavaScript.

```
@Component({
  selector: "sign-up-form",
  template: `
    <button (click)="increment()">Clique em mim</button>
    <p>Você clicou {{ count }} vezes.</p>
  `,
})
export class Count {
  count = 0;

  increment() {
    this.count++;
  }
}
```



Templates

Data Binding

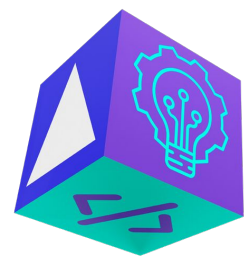
Pode acontecer por:

- **Event Binding**

\$event

Se você precisa acessar o Objeto Evento, Angular fornece uma variável `$event` que você pode passar para uma função.

```
<button (click)="createUser($event)">Submit</button>
```



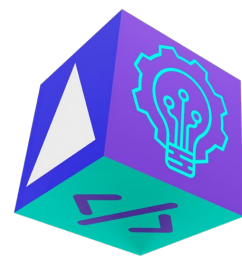
Templates

Condicionais e Loops

O Angular permite trabalhar com condicionais e loops no template.

Para isso, o Angular agora conta com as diretivas @if e @for.

```
@Component({
  standalone: true,
  selector: 'user-controls',
  template: `
    @if (isAdmin) {
      <button>Erase database</button>
    } @else {
      <p>You are not authorized.</p>
    }
  `,
})
export class UserControls {
  isAdmin = true;
}
```



Templates

Condicionais e Loops

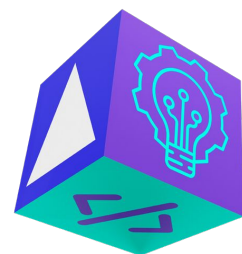
Semelhante ao que acontece na condicional, temos o `@for` para loops.

A propriedade track

Quando o Angular renderiza uma lista de elementos com `@for`, esses itens podem mudar ou mover-se posteriormente.

O Angular precisa rastrear cada elemento através de qualquer reordenamento, geralmente tratando uma propriedade do item como um identificador ou chave exclusiva.

```
@Component ({
  standalone: true,
  selector: "ingredient-list",
  template: `
    <ul>
      @for (ingredient of ingredientList; track ingredient.id) {
        <li>{{ ingredient.quantity }} - {{ ingredient.name }}</li>
      }
    </ul>`,
})
export class IngredientList {
  ingredientList = [
    { id: 1, name: "noodles", quantity: 1 },
    { id: 2, name: "miso broth", quantity: 1 },
    { id: 3, name: "egg", quantity: 2 },
  ];
}
```

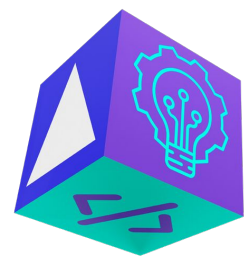


Pipes

Pipes são funções usadas para transformar dados no Templates.

Angular possui vários pipes integrados úteis que você pode importar e usar em seus componentes.

```
import { DatePipe, JsonPipe, UpperCasePipe } from
"@angular/common";
@Component({
  selector: "app-angular-pipes",
  standalone: true,
  imports: [DatePipe, UpperCasePipe, JsonPipe],
  template: `
    <ul>
      <li>{{ "Irlan Arley" | uppercase }}</li>
      <li>{{ jsonExample | json }}</li>
      <li>{{ newDate | date : "dd/MM/yyyy" }}</li>
    </ul>
  `,
})
export class AngularPipesComponent {
  public newDate = new Date();
  public jsonExample = {
    name: "John", age: 30, city: "New York" };
}
```



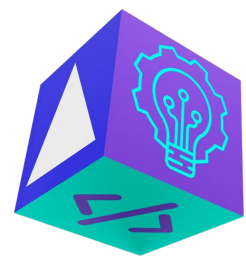
Pipes

Também é possível criar seus próprios pipes.

Comando: `ng generate pipe <nome do pipe>`

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'test',
  standalone: true
})
export class TestPipe implements PipeTransform {
  transform(value: unknown, ...args: unknown[]): unknown {
    return null;
  }
}
```



Pipes

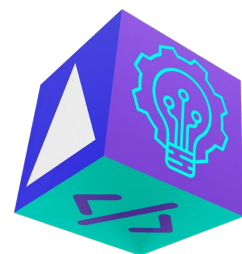
Também é possível criar seus próprios pipes.

Edite o arquivo para personalizar seu pipe.

```
@Pipe({
  name: 'customString',
  standalone: true,
})
export class CustomStringPipe implements PipeTransform {
  transform(value: string, args?: string): string {
    if (args === 'upper') return value.toUpperCase();
    return value;
  }
}
```

Template:

```
CustomString: {{ "irlan arley" | customString:"upper" }}
```



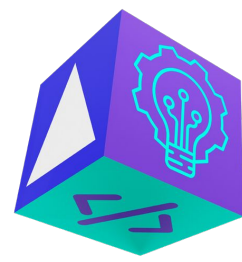
Comunicação Entre Componentes

Às vezes, o desenvolvimento de aplicativos exige o envio de dados para um componente. Esses dados podem ser usados para personalizar um componente ou enviar informações de um componente pai para um componente filho e vice-versa.

Comunicação de Pai para Filho

Angular usa um conceito chamado Input. Para criar uma propriedade Input, utilize o decorador **@Input**.

O Input é declarado na classe do componente, e seu valor é inserido no template.



Comunicação Entre Componentes

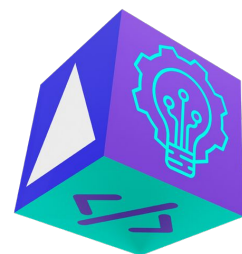
Comunicação de Pai para Filho

```
import { Component, Input } from '@angular/core';
```

```
@Component({
  selector: 'app-todolist',
  standalone: true,
  imports: [],
  template: `
    <div>
      <h1>{{ taskTitle }}</h1>
    </div>
  `,
})
export class TodoListComponent {
  @Input() taskTitle = '';
  isComplete = false;
}
```

```
import { Component } from '@angular/core';
import { TodoListComponent } from
'./components/todolist/todolist.component';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [TodoListComponent],
  template: `
    <div>
      <app-todolist taskTitle="Fazer compras" />
    </div>
  `,
})
export class AppComponent {
  title = 'projeto-angular';
}
```

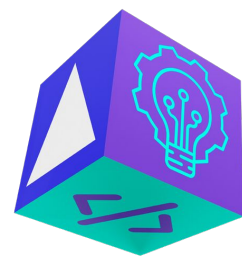


Comunicação Entre Componentes

Comunicação de Filho para Pai

Ao trabalhar com componentes, pode ser necessário notificar outros componentes de que algo aconteceu. Talvez um botão tenha sido clicado, um item tenha sido adicionado ou removido de uma lista, ou tenha ocorrido alguma outra atualização importante. Nesse cenário, os componentes precisam se comunicar com os componentes pai.

Angular usa o decorador `@Output` para habilitar esse tipo de comportamento.



Comunicação Entre Componentes

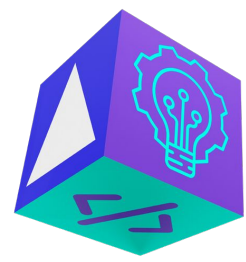
Comunicação de Filho para Pai

```
/* task-creator.component.ts */
import { Component, EventEmitter, Output }
from '@angular/core';
@Component({
  selector: 'app-task-creator',
  template: `
<button (click)="addTaskEvent('task')">Adicionar</button>
`,
})
export class TaskCreatorComponent {
  @Output() newTask = new EventEmitter<string>();

  addTaskEvent(taskTitle: string) {
    this.newTask.emit(taskTitle);
  }
}
```

```
/* app.component.ts */
@Component({
  ...
  templateUrl: `
    <app-task-creator (newTask)="addTask($event)" />
    <div>
      @for(task of todoList; track task){
        <app-todo-item [taskTitle]="task" />
      }
    </div>`,
})
export class AppComponent {
  todoList = Array<string>();

  addTask(taskTitle: string) {
    this.todoList.push(taskTitle);
  }
}
```



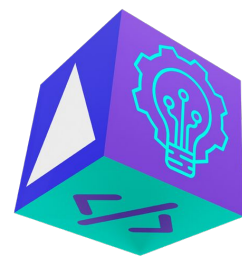
Formulários Reativos

Os formulários reativos proporcionam maior controle programático sobre o formulário.

Eles são ideais para formulários complexos e cenários que requerem uma lógica de formulário mais avançada.

Características

1. **Maior Controle e Escalabilidade:** A lógica do formulário é explicitamente definida no componente;
2. **Imutabilidade:** Utilizam FormControl, FormGroup e FormArray para gerenciar o estado do formulário;
3. **Validação Programática:** Validações são definidas programaticamente no componente;
4. **Observabilidade:** Fácil de observar e reagir a mudanças nos valores e no estado do formulário usando observáveis.



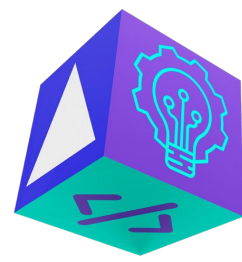
Formulários Reativos

Os formulários reativos usam a classe FormControl para representar os controles do formulário (por exemplo, entradas).

O Angular fornece a classe FormGroup para agrupar controles de formulário em um objeto, tornando o manuseio de formulários grandes mais conveniente para os desenvolvedores.

```
import { Component } from '@angular/core';  
import { ReactiveFormsModule, FormControl, FormGroup }  
from '@angular/forms';
```

```
@Component({  
  selector: 'app-formulario',  
  standalone: true,  
  imports: [ReactiveFormsModule],  
  templateUrl: './formulario.component.html',  
})  
export class FormularioComponent {  
  profileForm = new FormGroup({  
    name: new FormControl(''),  
    email: new FormControl(''),  
  });  
}
```



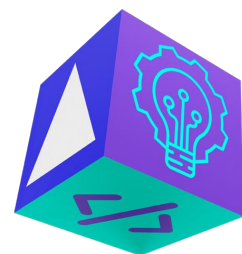
Formulários Reativos

Após a estrutura do componente estar pronta, é necessário fazer a ligação com o template.

Cada FormGroup deve ser anexado a um formulário usando a diretiva [formGroup].

Além disso, cada FormControl pode ser anexado utilizando a diretiva formControlName e atribuído à propriedade correspondente.

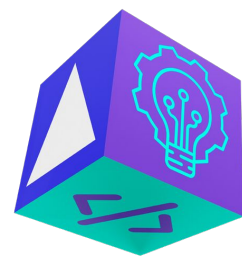
```
<form [formGroup]="profileForm">
  <label>
    Name
    <input type="text" formControlName="name" />
  </label>
  <label>
    Email
    <input type="email" formControlName="email" />
  </label>
  <button type="submit">Submit</button>
</form>
```



Formulários Reativos

Para acessar os valores inseridos nos inputs, basta acessar o FormGroup definido dentro do componente.

```
<h2>Profile Form</h2>  
<p>Name: {{profileForm.value.name}}</p>  
<p>Email: {{profileForm.value.email}}</p>
```



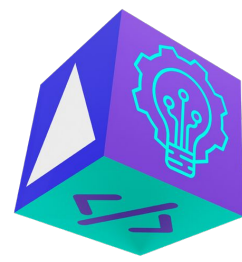
Formulários Reativos

É possível controlar o evento do formulário utilizando a diretiva `ngSubmit`.

Para isso, no componente, pode-se criar um método que será chamado quando o evento for disparado.

```
export class FormularioComponent {  
  ...  
  handleSubmit() {  
    alert(this.profileForm.value.name + ' ' +  
    this.profileForm.value.email);  
  }  
}
```

```
<form  
  [formGroup]="profileForm"  
  (ngSubmit)="handleSubmit()">  
  ...  
</form>
```

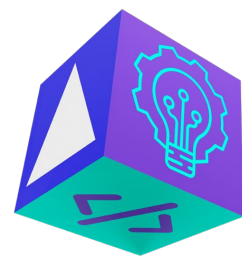


Formulários Reativos

Por fim, a validação dos formulários é feita utilizando a classe Validators, do angular/forms;

A validação pode ser inserida através de um array que é passado para o FormControl;

O FormControl fornece métodos que auxiliam das verificações de validação.



Formulários Reativos

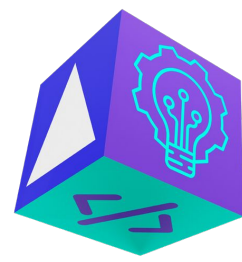
```
export class FormularioComponent {
  profileForm = new FormGroup({
    name: new FormControl('',
      [Validators.required]),

    email: new FormControl('',
      [Validators.required, Validators.email]),
  });

  handleSubmit() {
    if (this.profileForm.invalid) {
      alert('Formulario inválido');
      return;
    } else {
      alert('Formulario enviado: ' +
        this.profileForm.value.name +
        ' ' + this.profileForm.value.email);}}
  }
```

```
<form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
  <label>
    Name
    <input type="text" formControlName="name" />
    @if(profileForm.get('name')?.hasError('required')
      && profileForm.get('name')?.touched) {
      <span>Este campo é obrigatório</span>
    }
  </label>
  <br>
  <label>
    Email: <input type="email" formControlName="email" />
  </label>

  <button [disabled]="profileForm.invalid"
type="submit">Enviar</button>
</form>
```



Referências

- <https://angular.dev/>

Irlan Moreira

irlan.moreira@ifrn.edu.br

Wagner

irlan.moreira@ifrn.edu.br

